ETC: Exact Topological Computing

User Manual and API Reference
Version 0.1.0

Maitri Aniruddhbhai Savaliya
GSFC University, Vadodara, Gujarat, India
maitrisavaliya05@gmail.com

This document is the machine-readable user manual for the ETC Python library, submitted as supplementary material accompanying the ACM TOMS manuscript *ETC: Exact Topological Computing*. It describes all public constructors, methods, and parameters across the seven ETC modules.

Contents

1. Overview and Installation
2. etc.core.real -- ExactReal
3. etc.core.sign -- sign()
4. etc.certified -- Certified[V]
5. etc.verify.certificate -- Certificate types
6. etc.verify.formal -- Lean 4 Export
7. etc.topology -- Space, Path, Homotopy
8. etc.topology.invariants -- winding_number()
9. etc.geometry -- Curves, Polygon, Surfaces
10. etc.analysis -- Series, Calculus, ODE
11. etc.problems -- Problem registry
12. Timing API: timed_eval and benchmark_eval
13. Error handling and known limitations
14. Comparison script: run_comparison.py

1. Overview and Installation

ETC is a Python library providing exact real arithmetic, constructive topology, and machine-checkable proof certificates as a unified computational system. Its three primary abstractions are:

ExactReal -- an exact real number represented as a lazy Cauchy sequence seq: int -> Fraction satisfying |seq(n) - x| < 2^{-n}. All arithmetic is performed over Python's fractions.Fraction; no floating-point is used inside the Cauchy sequence machinery.

Certified[V] -- a typed wrapper pairing any discrete output (sign, winding number, orientation) with a machine-checkable Certificate. Every discrete result in ETC is a Certified object.

Lean4Exporter -- translates ETC certificates into Lean 4 theorem stubs checkable by the Lean kernel, bridging Python computation and formal proof.

Installation

```
# Standard install
pip install .

# With development tools (pytest, black, mypy, flake8)
pip install ".[dev]"

# Verify
python -c "import etc; print(etc.__version__)"  # 0.1.0

# Run all tests
pytest tests/ -v

# Run the float64 vs ETC comparison table (reproduces Table 1 of the paper)
python scripts/run_comparison.py

# Run the benchmark (reproduces Tables 3-6 of the paper)
python scripts/benchmark.py
```

Requirements: Python >= 3.9, sympy >= 1.12, mpmath >= 1.3.0. Lean 4 is required only for proof checking; it is not a Python package and must be installed separately (see INSTALL).

2. etc.core.real -- ExactReal

ExactReal represents a real number x as a lazy Cauchy sequence over Q: a function seq: int -> Fraction such that |seq(n) - x| < 2^{-n} for all n >= 0. Results are memoised by precision.

2.1 Constructors

Constructor	Description
ExactReal(seq)	From a raw SeqFn: int -> Fraction
ExactReal.from_rational(p, q=1)	Exact rational p/q. p may be int or Fraction
ExactReal.from_int(n)	Integer n (shorthand for from_rational(n))
ExactReal.zero()	The constant 0
ExactReal.one()	The constant 1
ExactReal.pi()	pi via Machin's formula. O(n^2 log n)
ExactReal.e()	e = sum 1/k!. O(n^2 log n)
ExactReal.log2()	ln(2). O(n^2 log n)
ExactReal.sqrt2()	sqrt(2) via Newton's method. O(n^2 log^2 n)
ExactReal.from_decimal(s)	Parse decimal string e.g. '3.14159' exactly
ExactReal.from_sympy(expr, sym, val)	Substitute val (ExactReal) into sympy expr

2.2 Evaluation

Method	Description
eval(n: int) -> Fraction	Return r in Q with |r - x| < 2^{-n}. Memoised: calling twice computes once. n >= 0 required.
timed_eval(n: int) -> (Fraction, float)	Evaluate and return (result, wall_seconds). Returns (cached, 0.0) on cache hit.
to_decimal(digits=20) -> str	Return decimal string with 'digits' significant figures.

2.3 Arithmetic

Method	Returns	Notes
add(other)	ExactReal	self + other
sub(other)	ExactReal	self - other
mul(other)	ExactReal	self * other; extra bits computed once at construction

div(other)	ExactReal	self / other; requires other != 0
neg()	ExactReal	-self
abs_val()	ExactReal	|self|
recip()	ExactReal	1/self; raises ZeroDivisionError if self is zero
pow_int(k: int)	ExactReal	self^k via binary exponentiation; k >= 0
pow_rational(p, q)	ExactReal	self^(p/q); dispatches to pow_int + Newton root
sqrt()	ExactReal	sqrt(self) via Newton. O(n^2 log^2 n)
cbrt()	ExactReal	cbrt(self) via Newton. O(n^2 log^2 n)

2.4 Transcendental functions

Method	Description
exp()	e^self with argument reduction; O(n^2 log n)
log()	ln(self) for self > 0; reduces to [1/2, 2) then arctanh series
sin()	sin(self); full argument reduction to [-pi/2, pi/2] then Taylor
cos()	cos(self); argument reduction to [-pi, pi] then Taylor
tan()	tan(self) = sin/cos
atan()	arctan(self); Gregory series with reduction to [0, 1/2]

2.5 Comparison

Method	Description
lt(other, prec=53) -> bool	self < other at given precision; raises ValueError if indistinguishable
gt(other, prec=53) -> bool	self > other
eq_approx(other, prec=53) -> bool	True if |self - other| < 2^{-prec} at prec bits
sign(prec=53) -> int	Quick sign in {-1, 0, 1} at prec bits (not certified; use etc.sign() for certified result)

2.6 Python operator overloads

ExactReal supports +, -, *, / with other ExactReal instances or with int/Fraction. float(x) calls x.eval(53) and converts to Python float. repr(x) displays the 53-bit float approximation.

2.7 Example

```
from etc import ExactReal
from fractions import Fraction

# Exact rational constant
a = ExactReal.from_rational(Fraction(1, 3))

# pi to 128 bits
pi_128 = ExactReal.pi().eval(128)   # returns exact Fraction

# Composed expression
x = a.add(ExactReal.pi()).mul(ExactReal.sqrt2())
print(x.to_decimal(30))  # 30-significant-digit decimal string

# Rump's polynomial (exact)
a_ = ExactReal.from_rational(77617)
b_ = ExactReal.from_rational(33096)
b2 = b_.mul(b_); b4 = b2.mul(b2); b6 = b4.mul(b2); b8 = b4.mul(b4)
a2 = a_.mul(a_)
result = (
    ExactReal.from_rational(Fraction(33375,100)).mul(b6)
    .add(a2.mul(
        ExactReal.from_rational(11).mul(a2).mul(b2)
        .sub(b6).sub(ExactReal.from_rational(121).mul(b4))
        .sub(ExactReal.from_rational(2))
    ))
    .add(ExactReal.from_rational(Fraction(11,2)).mul(b8))
    .add(a_.div(b_.mul(ExactReal.from_rational(2))))
)
print(result.eval(80))  # Fraction(-54767, 66192)
```

3. etc.core.sign -- sign()

sign() computes the sign of an ExactReal with a machine-checkable certificate. It evaluates the Cauchy sequence at increasing precision until the sign is unambiguous, then returns a Certified[int] with value in {-1, 0, +1}.

Signature

```
sign(x: ExactReal, max_prec: int = 128) -> Certified[int]
```

Parameters

Parameter	Type	Description
x	ExactReal	The number whose sign is to be determined
max_prec	int	Maximum precision bits to try (default 128). If the sign cannot be determined at this precision, 0 is returned with a note.

Returns

Certified[int] with .value in {-1, 0, +1}. The .proof attribute is a Certificate recording the precision at which certainty was achieved and the approximant used.

Algorithm

Evaluate x.eval(n) for n = 8, 16, 32, ..., max_prec. At each precision n, if |approx| > 2^{-(n-1)}, the sign is certain. If no precision settles it, return 0. Termination guarantee: for x != 0 with |x| >= 2^{-k}, the algorithm terminates at prec = max(2k+4, 8).

Example

```
from etc import ExactReal, sign

x = ExactReal.pi().sub(ExactReal.from_rational(3))
result = sign(x)

print(result.value)            # 1
print(result.is_valid())       # True
print(result.proof.summary())  # Certificate [VALID]
                               #   Claim  : sign(x) = +1
                               #   Method : cauchy_evaluation(prec=8)

# Serialise to JSON
import json
d = json.loads(result.proof.to_json())
print(d['evidence']['prec_bits'])  # 8
```

4. etc.certified -- Certified[V]

Certified[V] is a generic typed wrapper pairing a computed value with its proof certificate. Every discrete output in ETC (sign, winding_number, orientation) is a Certified object.

Constructor

```
Certified(value: V, proof: Certificate)
```

Attributes and methods

Attribute/Method	Type	Description
.value	V	The computed result (e.g. int for sign or winding_number)
.proof	Certificate	Machine-checkable evidence. Use .proof.summary() to inspect.
.is_valid() -> bool	bool	Re-run the proof check from stored evidence
== other	bool	Compares .value only (not the proof)
int(cert)	int	Converts .value to int (useful for winding numbers)

Example

```
from etc import sign, ExactReal

result = sign(ExactReal.pi().sub(ExactReal.from_rational(3)))
print(result)         # Certified(value=1, claim='sign(x) = +1', VALID)
print(result.value)   # 1
print(int(result))    # 1
print(result == 1)    # True  (compares value only)
```

5. etc.verify.certificate -- Certificate types

Every ETC computation producing a discrete result generates a Certificate. Certificates are self-contained, serialisable to JSON, and re-checkable independently of the original computation.

5.1 Base Certificate

Fields:

Field	Type	Description
claim	str	Human-readable statement of what is proved
method	str	Proof technique (cauchy_evaluation, symbolic, spot_check, etc.)
valid	bool	True iff all checks passed at creation time
evidence	dict	Serialisable dictionary of proof evidence
timestamp	float	Unix time when certificate was created

Methods:

Method	Description
is_valid() -> bool	Re-run checks from stored evidence
summary() -> str	Human-readable multi-line summary
to_json() -> str	Serialise to JSON string

5.2 Certificate subtypes

Subtype	Description
PointCertificate	Proves P in Space. evidence: coordinates, prec, space_name
PathCertificate	Proves gamma: [0,1] -> Space. evidence: n_spot_checks, params checked
HomotopyCertificate	Proves H: [0,1]^2 -> Space with endpoint conditions
IdentityCertificate	Proves f = 0 as a symbolic identity via SymPy trigsimp/simplify
ComposedCertificate	Conjunction of sub-certificates; is_valid() returns AND of all sub-checks

5.3 Factory functions

Function	Returns	Description
certify_point(point, space, prec=40)	PointCertificate	Check point in space via predicate at prec bits

certify_path(path, prec=40, n_spot=10)	PathCertificate	Spot-check path at n_spot rational parameter values
certify_homotopy(homotopy, prec=40, n_spot=5)	HomotopyCertificate	Spot-check H on n_spot x n_spot grid
certify_identity(lhs, rhs, variables=None)	IdentityCertificate	SymPy simplify(lhs - rhs) == 0
compose(*certs)	ComposedCertificate	Conjoin multiple certificates

5.4 CertificateStore

A named dictionary of certificates for batch management:

```
from etc.verify.certificate import CertificateStore, certify_identity
import sympy as sp

x = sp.Symbol('x')
store = CertificateStore()
store.add('pythagorean', certify_identity(sp.cos(x)**2 + sp.sin(x)**2, 1, [x]))
print(store.all_valid())  # True
print(len(store))         # 1
cert = store.get('pythagorean')
```

6. etc.verify.formal -- Lean 4 Export

The Lean4Exporter translates ETC certificates into Lean 4 source code targeting Mathlib4. Algebraic identities proved by ring or norm_num are fully proved. Transcendental or continuity properties receive sorry stubs with explicit OPEN OBLIGATION labels and proof strategies.

Lean4Exporter

```
exporter = Lean4Exporter()
exporter.export_point_certificate(cert, name='myPoint')
exporter.export_path_certificate(cert, name='myPath')
exporter.export_identity_certificate(cert, name='pythagorean')
lean_src = exporter.render()   # str: Lean 4 source code
exporter.write('output.lean') # write to file
```

certificate_to_lean() convenience function

```
from etc.verify.formal import certificate_to_lean
from etc.verify.certificate import certify_identity
import sympy as sp

x = sp.Symbol('x')
cert = certify_identity(sp.cos(x)**2 + sp.sin(x)**2, sp.Integer(1), [x])
src = certificate_to_lean(cert, name='pythagorean')
print(src)
# import Mathlib
# theorem pythagorean : ...
#   exact Real.cos_sq_add_sin_sq x
```

Open proof obligations

Four proof obligations are marked sorry in the generated output. Each sorry carries an OPEN OBLIGATION comment with the precise statement and a suggested proof strategy:

Obligation	Strategy
Path membership continuity: forall t in [0,1], gamma(t) in Space	Lean ContinuousOn tactics with interval-arithmetic bounds
Winding number arctan2 bound	Mathlib Real.arctan lemmas with Arb interval certificates
Homotopy endpoint conditions: H(0,t) = gamma_0(t) and H(1,t) = gamma_1(t)	Density arguments at rational parameters; or ring/norm_num if symbolic
Pythagorean identity	exact Real.cos_sq_add_sin_sq x (one-line Mathlib proof)

7. etc.topology -- Space, Path, Homotopy

7.1 Space

```
Space(
    predicate:    Callable[[Coords], bool],
    dim:          int,
    sympy_pred  = None,   # sympy.Expr: f(coords) = 0 on Space
    sympy_coords= None,   # Tuple[sympy.Symbol, ...]
    name:  str = 'Space',
    description: str = '',
)
```

Method	Description
contains(coords: Coords) -> bool	Check membership via exact predicate
verify_path_symbolic(path_exprs, param_sym) -> bool	Symbolically prove path stays in Space for all parameter values

Factory functions for common spaces:

Function	Description
make_Lp_circle(p: float) -> Space	L^p 'circle' {(x,y) | |x|^p + |y|^p = 1} in R^2
make_sphere(r=1) -> Space	Unit sphere in R^3
make_torus(R, r) -> Space	Torus with major radius R and minor radius r

7.2 UnitInterval

UnitInterval wraps an ExactReal constrained to [0, 1]. Construction raises ValueError if the value is outside [0, 1].

```
UnitInterval(x: ExactReal, _skip_check=False)
UnitInterval.from_rational(p, q=1) -> UnitInterval
UnitInterval.zero()  -> UnitInterval   # t = 0
UnitInterval.one()   -> UnitInterval   # t = 1
UnitInterval.half()  -> UnitInterval   # t = 1/2
t.complement()       -> UnitInterval   # 1 - t
```

7.3 Path

```
Path(
    path_fn:      Callable[[UnitInterval], Coords],
    space:        Space,
    sympy_path  = None,   # Tuple[sympy.Expr, ...] in param_sym
    sympy_param = None,   # sympy.Symbol
    name: str   = 'gamma',
```

)
```

| Method | Description |
|---|---|
| evaluate(t: UnitInterval) -> Coords | Evaluate path at parameter t |
| spot_check(n: int = 10, prec: int = 40) -> bool | Check membership at n equally-spaced rational parameter values |
| symbolic_verify() -> bool | Symbolically prove path stays in Space (requires sympy_path and sympy_param) |
| is_closed(prec: int = 40) -> bool | Check gamma(0) == gamma(1) within 2^{-prec} |

## 7.4 Homotopy

```
Homotopy(
    homotopy_fn:    Callable[[UnitInterval, UnitInterval], Coords],
    space:          Space,
    path0:          Path,   # H(0, .) = path0
    path1:          Path,   # H(1, .) = path1
    sympy_homotopy: Optional[Tuple] = None,
    s_sym  = None,
    t_sym  = None,
    name:   str = 'H',
)
```

Method	Description
spot_check(n: int = 5, prec: int = 40) -> bool	Check H on n x n parameter grid
verify_endpoints(prec=40) -> bool	Check H(0,t) = path0(t) and H(1,t) = path1(t) at spot points
symbolic_verify() -> bool	Full algebraic proof via SymPy
verify() -> bool	Run all three verification steps
```

8. etc.topology.invariants -- winding_number()

```
winding_number(
    path:        Path,
    base_point:  Optional[Tuple[ExactReal, ExactReal]] = None,
    prec:        int = 53,
    n_steps:     int = 200,
) -> Certified[int]
```

Parameters

Parameter	Type	Description
path	Path	A closed path in R^2 (space must be 2-dimensional)
base_point	(ExactReal, ExactReal) or None	The point to wind around. Defaults to (0, 0) if None.
prec	int	Working precision in bits for angle computations (default 53)
n_steps	int	Number of angle samples. Higher values reduce rounding error but increase runtime. Default 200; paper uses 400.

Returns

Certified[int] with .value = winding number in Z. The .proof Certificate includes rounding_error (accumulated arctan2 error) and path_closed_gap (|gamma(0) - gamma(1)|).

Raises

ValueError if the path passes through the base point (angle undefined), or if the path is not closed (gamma(0) != gamma(1) within tolerance).

Example

```
import math
from fractions import Fraction
from etc import ExactReal, winding_number
from etc.topology.space import make_Lp_circle
from etc.topology.path import Path, UnitInterval

# Build a circular path winding once around the origin
space = make_Lp_circle(2)

def gamma(t: UnitInterval):
    theta = float(t.value.eval(20)) * 2 * math.pi
    return (
        ExactReal.from_rational(Fraction(math.cos(theta)).limit_denominator(10**9)),
        ExactReal.from_rational(Fraction(math.sin(theta)).limit_denominator(10**9)),
    )

path = Path(gamma, space, name='circle_1x')
```

```
result = winding_number(path, n_steps=400)

print(result.value)          # 1
print(result.is_valid())     # True
print(result.proof.evidence['rounding_error'])  # ~4.44e-16
```

9. etc.geometry -- Curves, Polygon, Surfaces

9.1 AlgebraicCurve

```
AlgebraicCurve(sympy_poly, x_sym, y_sym, name='curve')

# Methods
.as_space(check_prec=40) -> Space   # build Space from polynomial predicate
.point(x, y)             -> Point   # construct and certify a point on the curve
.tangent_vector(pt, prec=30) -> (float, float)   # approximate tangent direction
```

9.2 ParametricCurve

```
ParametricCurve(
    x_fn: Callable[[ExactReal], ExactReal],
    y_fn: Callable[[ExactReal], ExactReal],
    name: str = 'curve',
)

.evaluate(t: ExactReal) -> (ExactReal, ExactReal)
.as_path(space, name) -> Path
```

9.3 ExactPolygon

```
ExactPolygon(vertices: List[Tuple[ExactReal, ExactReal]])

# Requires at least 3 vertices
.n_vertices() -> int
.signed_area(prec=60) -> ExactReal   # exact via Shoelace formula
.area(prec=60)        -> ExactReal
.centroid(prec=60)    -> (ExactReal, ExactReal)
.is_convex(prec=40)   -> bool
.winding_number(pt, prec=53) -> int   # exact integer
.contains_point(pt, prec=53) -> bool  # True if winding != 0
```

9.4 AlgebraicSurface

```
AlgebraicSurface(sympy_poly, x_sym, y_sym, z_sym, name='surface')

.as_space(check_prec=40) -> Space
.point(x, y, z)          -> Point
.normal_vector(pt, prec=30) -> (float, float, float)   # unit normal grad f / |grad f|
```

10. etc.analysis -- Series, Calculus, ODE

The analysis module provides power series, symbolic calculus, and ODE tools built on top of ExactReal and SymPy.

10.1 PowerSeries and TaylorSeries

```
# PowerSeries: sum_{k=0}^{N} c_k * x^k
from etc.analysis.series import PowerSeries, TaylorSeries

coeffs = [ExactReal.from_rational(c) for c in [1, 0, -1, 0, 1]]
ps = PowerSeries(coeffs)
val = ps.evaluate(ExactReal.from_rational(Fraction(1, 2)), prec=60)

# TaylorSeries: expand a SymPy expr around x=a to order n
import sympy as sp
x = sp.Symbol('x')
ts = TaylorSeries(sp.sin(x), x, center=0, order=10)
approx = ts.to_power_series()   # -> PowerSeries
```

10.2 Calculus

```
from etc.analysis.calculus import Calculus

calc = Calculus()
# Symbolic derivative
x = sp.Symbol('x')
df = calc.differentiate(sp.sin(x)**2, x)   # 2*sin(x)*cos(x)

# Numerical integration via rectangle rule on ExactReal grid
integrand = lambda t: t.mul(t)   # f(t) = t^2
result = calc.integrate(integrand, a=0, b=1, n_intervals=100, prec=40)
# result is ExactReal; float(result.eval(40)) ~ 0.3333...
```

10.3 ODE

```
from etc.analysis.ode import ODE

# Solve y' = f(t, y) with initial condition y(0) = y0
# using exact Euler method on rational grid
ode = ODE(
    f=lambda t, y: y,              # y' = y  (solution: e^t)
    y0=ExactReal.from_rational(1), # y(0) = 1
    t_span=(0, 1),
    n_steps=100,
    prec=60,
)
sol = ode.solve()   # -> ODESolution
print(float(sol.y_at(1).eval(40)))  # approx e ~ 2.71828...
```

11. etc.problems -- Problem registry

The problems module provides a structured interface for the canonical benchmark problems implemented in ETC. Each problem class implements solve() -> Certified and verify(answer) -> bool. To run all seven problems and print a formatted comparison table matching Table 1 of the paper, use scripts/run_comparison.py.

Problem abstract base class

```
from etc.problems.base import Problem

class Problem(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def description(self) -> str: ...

    @abstractmethod
    def solve(self) -> Certified[Any]: ...

    @abstractmethod
    def verify(self, answer: Any) -> bool: ...
```

ProblemRegistry

```
from etc.problems.registry import registry

registry.register(RumpPolynomial())
registry.list()                    # prints all problem names
p = registry.get('rump_polynomial')  # fetch by name
result = p.solve()                 # Certified answer
print(p.verify(result.value))      # True
```

Canonical problems (paradigm_shift module)

Class	Problem	Exact answer
RumpPolynomial	f(77617, 33096) -- Rump 1988	Fraction(-5476
L5WindingNumber	Winding number of L^5 circle path around origin	1 (certified)

12. Timing API: timed_eval and benchmark_eval

ETC provides two timing interfaces following the Johnson (2002) methodology recommended by ACM TOMS.

12.1 timed_eval()

```
# Instance method on ExactReal
val, t = ExactReal.pi().timed_eval(128)
# val: Fraction  -- exact rational at 128-bit precision
# t:   float     -- wall-clock seconds (0.0 on cache hit)

# Example output:
print(f"pi ~ {float(val):.15f}  [{t*1000:.2f} ms]")
# pi ~ 3.141592653589793  [0.39 ms]
```

12.2 benchmark_eval()

```
from etc import ExactReal, benchmark_eval

# benchmark_eval(x, precisions, repeat=3) -> List[(int, float)]
# Creates a FRESH ExactReal instance (empty cache) per precision.
# Returns the MINIMUM wall-clock time over 'repeat' runs.

results = benchmark_eval(
    ExactReal.pi(),
    precisions=[8, 16, 32, 64, 128, 256, 512, 1024],
    repeat=3,
)
for n, t in results:
    print(f"n={n:5d}  {t*1000:.3f} ms")
```

Note on the pre-asymptotic regime

At precision levels n <= 1024 bits, the measured scaling exponents are below the theoretical O(n^2 log n) value of 2.0 because Python interpreter overhead and the constant-factor cost of Fraction GCD dominate. Reliable asymptotic exponent fitting requires n >= 4096. The benchmark script (scripts/benchmark.py) reports fitted exponents for each doubling.

13. Error handling and known limitations

13.1 Exceptions

Exception	Raised when
ValueError	precision < 0 in eval() or timed_eval(); ExactReal argument out of domain (e.g. log of negative, sqrt of negative); path not closed in winding_number(); base_point on path in winding_number()
ZeroDivisionError	recip() or div() called when denominator is zero or converges to zero faster than the precision search can establish |x| > 0
ImportError	SymPy not installed when using etc.verify.symbolic or certify_identity()

13.2 Known limitations

Trigonometric argument reduction for large inputs. sin() and cos() reduce modulo 2*pi using round(float(x / 2*pi)), which introduces a float rounding error of up to 2^{-52} relative to the true quotient. For inputs where x / 2*pi is within 2^{-52} of a half-integer, the wrong multiple may be selected. In practice this is not triggered for |x| <= 10^3 and n <= 512 bits, but users should not rely on sin() or cos() for very large arguments (|x| >> 2^{52}) without independent range reduction.

from_sympy() uses mpmath internal API. ExactReal.from_sympy() accesses mpmath's _mpf_ internal attribute to convert symbolic expressions. This may break on future mpmath versions. For production use, prefer ExactReal.from_rational() with a pre-computed rational approximant.

recip() iteration limit. recip() searches up to k=300 to establish |self| > 0. For values that converge to zero very slowly this limit may be reached, raising ZeroDivisionError even for non-zero inputs. Increase the limit by subclassing ExactReal if needed.

Lean 4 sorry stubs. Four proof obligations remain as sorry stubs in Lean 4 export: path membership continuity, winding number arctan2 bounds, homotopy endpoint conditions universally, and the Pythagorean identity. Each sorry carries an OPEN OBLIGATION comment with the precise statement and a suggested proof strategy. See Section 6 for details.

Performance. Exact arithmetic over Fraction is substantially slower than float64: pi at 512 bits takes approximately 1.7 ms versus nanoseconds for a float division. ETC is suitable where correctness is paramount and the number of exact operations is moderate. It is not suitable for high-throughput numerical simulation or large-scale matrix operations.

> End of ETC User Manual. For the theoretical background and experimental results, see the accompanying ACM TOMS manuscript.

14. Comparison script: run_comparison.py

The script scripts/run_comparison.py runs all seven canonical floating-point failure cases and prints a formatted Float64 vs ETC comparison table that directly reproduces Table 1 of the accompanying

paper. All ETC results are computed live by running the library; no values are hardcoded in the script.

Usage

```
python scripts/run_comparison.py
```

Expected output

```
Float64 vs ETC -- Seven Canonical Floating-Point Failure Cases
============================================================
#   | Problem                          | float64 result                | ETC result
-------------------------------------------------------------------------------------------
1   | Rump f(77617,33096)              | -1.180592e+21                 | -54767/66192 = -0.8
2   | 10^16 + 1 - 10^16                | 0.0  (should be 1)            | 1  (exact integer)
3   | (sqrt(x+1)-sqrt(x))*(...)-1      | 0.1780  (should be 0)         | |f| = 1.32e-85; pro
4   | (x-1)^10 at x=1+1e-6             | 4.66e-15 (s/b 1e-60)          | 1/10^60 (exact)
5   | Orientation eps=1e-16            | cross=0.0 (collinear)         | sign=+1 CCW cert_va
6   | Winding number (L5 curve)        | 1.000000 (uncertified)        | 1  (exact int, cert
7   | cos^2(x)+sin^2(x)=1              | max err=2.22e-16              | proved for all x in

Summary: float64 passed 0/7 problems | ETC passed 7/7 problems
```

What each column means

Column	Description
#	Problem number matching Table 1 of the paper
Problem	Short label for the canonical failure case
float64 result	The value returned by Python float / NumPy float64. Includes a note on what the correct answer should be.
ETC result	The exact value computed by ETC, shown as a Fraction or symbolic proof result. cert_valid=True confirms the Certificate.is_valid() check passes.
Float	PASS if float64 gives the correct answer (certifiably); FAIL otherwise. All seven problems are FAIL for float64.
ETC	PASS if ETC gives the correct certified answer; FAIL otherwise. All seven problems are PASS for ETC.

What each problem tests

#	Failure mode tested
1	Catastrophic cancellation: terms reach 10^36, sum is O(1)
2	Absorption: 10^16 + 1 rounds to 10^16 in float64
3	Difference-of-squares identity: float loses all significant digits

4	Polynomial sensitivity near a root: wrong sign and magnitude
5	Orientation predicate: cross-product underflows to zero
6	Winding number: float cannot certify the integer result
7	Pythagorean identity: float cannot prove for all x in R

Runtime

The script takes approximately 10-30 seconds depending on hardware. Problem 6 (L5WindingNumber) is the slowest step, building a 200-vertex exact polygon and computing the winding number via ExactPolygon. No additional dependencies beyond the base install are required.

Relationship to the test suite

The test suite (pytest tests/ -v) verifies correctness via assertions. The comparison script is complementary: it prints human-readable output suitable for a reviewer to read directly, showing the float64 failure and ETC success side by side for each problem.